FR

Bienvenue sur Sakata

Ce portail est un sanctuaire numérique dédié à la préservation et à la transmission de l'identité culturelle du peuple **Sakata**.

Notre objectif est d'unir nos racines ancestrales avec la splendeur numérique moderne, en offrant à la fois des archives vivantes, des sagesses intemporelles et un forum d'échange intergénérationnel.

Suivant →

> *La messagerie privée de Sakata est une expérience proche de WhatsApp/Signal : conversations 1-à-1, messages texte, vocaux, images éphémères, indicateurs de frappe en direct. Sa fiabilité repose sur quelques patterns critiques que tout changement futur doit respecter.*

DÉFINITION

Qu'est-ce que c'est ?

Accessible depuis `/chat`, la messagerie permet à deux utilisateurs authentifiés d'échanger en temps réel. Les fonctionnalités principales :

- Messages texte avec emoji, longueur illimitée pratique.
- Enregistrement audio avec aperçu pré-envoi (style WhatsApp).
- Images éphémères : visibles 1 ou 2 fois puis verrouillées.
- Mode conversation temporaire (24h ou 48h, expiration auto).
- Indicateur de frappe (typing) avec broadcast Realtime.
- Statut de lecture par message (vu/non vu).
- Détection de capture d'écran sur images éphémères (heuristique).

VUE D'ENSEMBLE

Architecture

Tables Supabase

TABLE	RÔLE
`chat_conversations`	Conversation (id, créée_at, type, ephemeral_until)
`chat_participants`	Lien M:N conversation ↔ user (lu_at, dernier accès)
`chat_messages`	Messages (texte, attachments, expires_at, max_views)
`chat_reactions`	Réactions emoji par message
`chat_typing`	Présence de frappe (broadcast-only, pas persisté)

Composants React

- `ChatSidebar` — Liste des conversations, badges non-lus.
- `ChatWindow` — Fenêtre active, header dynamique.
- `ChatInput` — Saisie : texte, audio, image éphémère.
- `MessageBubble` — Rendu adaptatif par type de message.
- `EphemeralImagePicker` — Sélecteur de mode d'image (Normal / 1× / 2×).

Hooks

- `useConversations` — Liste des conversations + Realtime.
- `useMessages` — Messages d'une conversation + Realtime + pagination.
- `useTyping` — Broadcast d'indicateur de frappe.

ARCHITECTURE

Pattern critique : anti-boucle de subscription

BUG SUBTIL À ÉVITER À TOUT PRIX

Une mauvaise gestion du `setLoading` dans le callback d'une subscription Supabase crée une **boucle infinie de spinner** visible par l'utilisateur. Symptômes : page qui clignote en permanence, impossibilité d'interagir.

PATTERN CORRECT (USECONVERSATIONS.TS) TYPESCRIPT

```typescript
async function fetchConversations(showLoading = false) {
  if (isFetchingRef.current) return;   // 1. Garde concurrence
  isFetchingRef.current = true;
  if (showLoading) setLoading(true);   // 2. Spinner SEULEMENT au montage initial
  try {
    // ... fetch
  } finally {
    isFetchingRef.current = false;
    if (showLoading) setLoading(false);
  }
}

// Montage initial : oui, on veut le spinner
fetchConversations(true);

// Callback realtime : NON, surtout pas
channel.on("postgres_changes", ..., () => {
  fetchConversations(false);  // silent refetch
});
```

ANTI-PATTERN À PROSCRIRE

CODE TYPESCRIPT

```typescript
// ❌ NE JAMAIS FAIRE
channel.on("postgres_changes", ..., () => {
  setLoading(true);   // déclenche un re-render
  fetchConversations();  // qui se subscribe à nouveau
  // ⟶ boucle infinie
});
```

SOUS-FONCTIONNALITÉ

Images éphémères

Trois modes d'envoi d'image existent : **normal** (pas d'expiration), **vue 1×** (visible une fois puis verrouillée), et **vue 2×**. Implémenté via la colonne `max_views` sur `chat_messages` et un compteur côté client persisté en localStorage.

Règle absolue : l'envoyeur voit toujours sa propre image

Le bug initial : l'envoyeur lui-même voyait son image se verrouiller après avoir cliqué. Correction : vérifier `isMe` avant d'appliquer `viewState = "locked"`.

CODE TYPESCRIPT

```typescript
if (!isMe && viewedCount ≥ maxViews) {
  setViewState("locked");
}
```

Persistence de l'état vu

La clé localStorage est `msgViewedKey(id)` → `"sakata-msg-viewed-{id}"`. Le préfixe `sakata-` est obligatoire pour respecter la whitelist du AuthProvider — sans cela, la clé serait purgée au prochain bump de `APP_VERSION`.

Détection de capture d'écran

Le navigateur ne peut pas vraiment détecter les screenshots, mais on peut intercepter quelques signaux :

- `visibilitychange` + `window.blur` → l'utilisateur a basculé vers un autre onglet/app, suspect.
- Application immédiate d'un overlay flou pendant l'absence de focus.
- Skippé si `isMe` (l'envoyeur peut tout faire avec sa propre image).

> **LIMITE RECONNUE**
>
> Aucune méthode 100% fiable contre les screenshots côté navigateur. C'est une mesure dissuasive, pas une garantie cryptographique.

RETOUR D'EXPÉRIENCE

Difficultés rencontrées

Stale closure sur user.id dans les subscriptions

Pattern courant : un `useEffect` qui crée un channel Supabase et utilise `userId` dans son callback. Quand l'utilisateur change (rotation token, switch d'onglet), le channel garde l'ancien `userId` en closure → bugs subtils.

CODE — TYPESCRIPT

```typescript
// ✅ Solution : ref synchronisée
const userIdRef = useRef(userId);
useEffect(() => {
  userIdRef.current = userId;
}, [userId]);

useEffect(() => {
  const channel = supabase.channel( ... ).on( ... , () => {
    // Toujours la valeur fraîche
    if (userIdRef.current) { ... }
  });
}, []);
```

WebSocket déconnecté silencieusement

Une déconnexion réseau (mode veille, changement de Wi-Fi) ferme le WebSocket Supabase. Sans handler d'erreur, c'est **invisible** côté UI. Tous les `.subscribe()` doivent inclure :

CODE — TYPESCRIPT

```typescript
channel.subscribe((status, err) => {
  if (status === "CHANNEL_ERROR" || err) {
    console.error("[Chat] Channel error:", err);
    // Trigger reconnect ou toast utilisateur
  }
});
```

Saturation de la file de requêtes

Le proxy Supabase de Sakata limite à 4 requêtes Postgrest simultanées (voir `src/lib/supabase.ts`). Sans `AbortController` sur les `useEffect` de fetch, les sockets restent occupées trop longtemps après démontage.

À SAVOIR

Limitations connues

- Conversations 1-à-1 uniquement (pas de groupes pour l'instant).
- Pas de chiffrement end-to-end — Supabase a accès en clair.
- Audio limité à ce que `MediaRecorder` produit (codec dépend du navigateur).
- Pas de transcription automatique des messages vocaux (prévu via Whisper dans la roadmap Veillée Numérique).

Sakata.com — Documentation interne · feature-chat-messaging · v2026.4

« Moto oyo ayebi te kotombola miso na likolo, akoki te koyeba esika oyo mbula eutaka. » — Proverbe Sakata

SAKATA

© 2026 — Brume de la Rivière. Tous droits réservés.

CULTURE ECOLE LANGUE HISTOIRE FORUM